



Tecnológico de Monterrey

Campus Guadalajara

Implementación de un procesador RISC-V single cycle

Marco Natsumi Rabiela Mun
A01647377

Gonzalo Gutierrez Ramos
Diseño de Sistemas en Chip
Grupo 501

28 de mayo del 2026

1. Resumen

Este reporte técnico muestra cómo diseñamos y conectamos los módulos de un procesador RISC-V (RV32I) de ciclo único. Para comprobar que todo funciona bien, corrimos una simulación en ModelSim cargando un programa de prueba real con 20 instrucciones (sumas, restas, cargas de memoria y saltos). Usamos un testbench para darle los pulsos de reloj y verificar paso a paso cómo responde el circuito.

2. Descripción Estructural de los Módulos

El procesador se compone de una serie de bloques interconectados de manera estructural a través del módulo RISC_V_single_cycle. A continuación, se detalla la función matemática y lógica de cada componente:

2.1. ProgramCounter (PC)

Registro síncrono de 32 bits sensible al flanco de subida del reloj (clk). Su función principal es almacenar la dirección de memoria de la instrucción en ejecución. Cuenta con una línea de reinicio (rst) que fuerza el estado inicial a PC = 0x00000000..

2.2. InstructionMemory (ROM)

Es una memoria que solo sirve para leer y tiene espacio para guardar 64 instrucciones (cada una de 32 bits). Como en RISC-V las instrucciones siempre miden 4 bytes y están alineadas, el circuito ignora los últimos dos bits del bus de direcciones (A[31:2]). Esto sirve para convertir la dirección de bytes del PC (que va saltando de 4 en 4: 0, 4, 8, 12...) en la posición exacta dentro de la memoria (0, 1, 2, 3...). Por último, usa el comando \$readmemh al arrancar para cargar automáticamente todo el código máquina que escribiste en el archivo externo programa.mem.

2.3. Register_File (Banco de Registros)

Es el bloque que guarda los datos con los que trabaja el procesador. Adentro tiene un arreglo de 32 registros, y cada uno puede almacenar un número de 32 bits.

Funciona con tres "puertas" de acceso al mismo tiempo:

- **Dos puertos de lectura asíncronos:** Usan las direcciones A1 y A2 para sacar los datos de dos registros de forma inmediata (sin esperar al reloj) y entregarlos por las salidas RD1 y RD2.
- **Un puerto de escritura síncrono:** Usa la dirección A3 para saber dónde guardar un dato. Este puerto solo se activa si la señal de control RegWrite (o

WE3) está en 1, y el dato se guarda físicamente justo cuando el reloj hace un flanco de subida.

2.4. ControlUnit (Unidad de Control)

Bloque combinacional encargado de decodificar el opcode (RD[6:0]), funciones (funct3 y funct7), e integrar el estado de las banderas de la ALU (Zero). Controla las señales de control del datapath: RegWrite, ImmSrc, AluSrc, MemWrite, ResultSrc, AluControl y PCSrc.

2.5. Alu (Unidad Aritmético Lógica)

Realiza operaciones aritméticas y lógicas según la señal de control AluControl. Y también genera el valor de la bandera zero.

2.6. DataMemory (RAM)

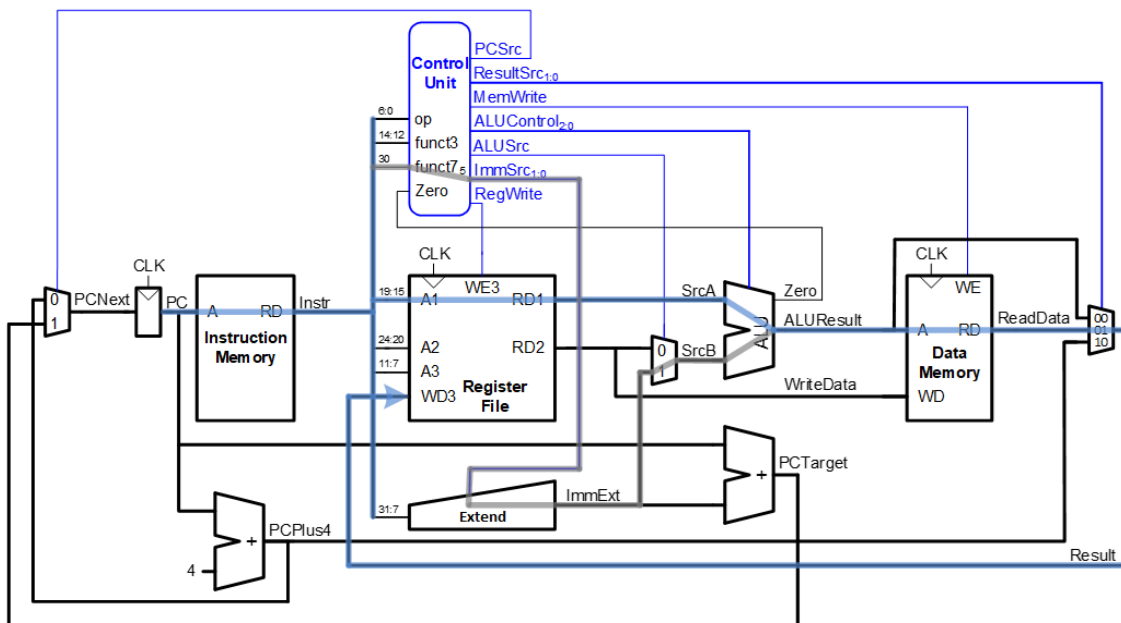
Bloque de memoria encargado de la persistencia de datos. Controlado por la señal combinacional MemWrite; si está activa, en el flanco de subida del reloj escribe el contenido de WriteData en la celda apuntada por Alu_result. De lo contrario, lee el contenido hacia el bus ReadData.

2.7. Bloques Auxiliares (Adder, Imm_Extend, Mux)

- **Adder (Sumadores):** Unidades dedicadas que realizan sumas aritméticas puras de 32 bits sin signo. Se emplean de forma estática para calcular $PC + 4$ y $PC + ImmExt$.
- **Imm_Extend:** Unidad de extensión de signo combinacional que extrae e infla los bits de los valores inmediatos codificados dentro de la propia instrucción dependiendo de su formato de codificación (Tipo I, S, B, etc.).
- **Mux (Multiplexores):** Selectores combinacionales multi-canal encargados de conmutar y enrutar las señales de datos correctas basándose en las líneas de selección emitidas por la Unidad de Control.

3. Diagrama del Camino de Datos Completo (Datapath)

A continuación, se presenta la representación esquemática del flujo estructural de datos y control. Las líneas negras continuas denotan buses de datos de 32 bits, mientras que las líneas punteadas azules indican señales de control combinacional:



4. Programa de Prueba

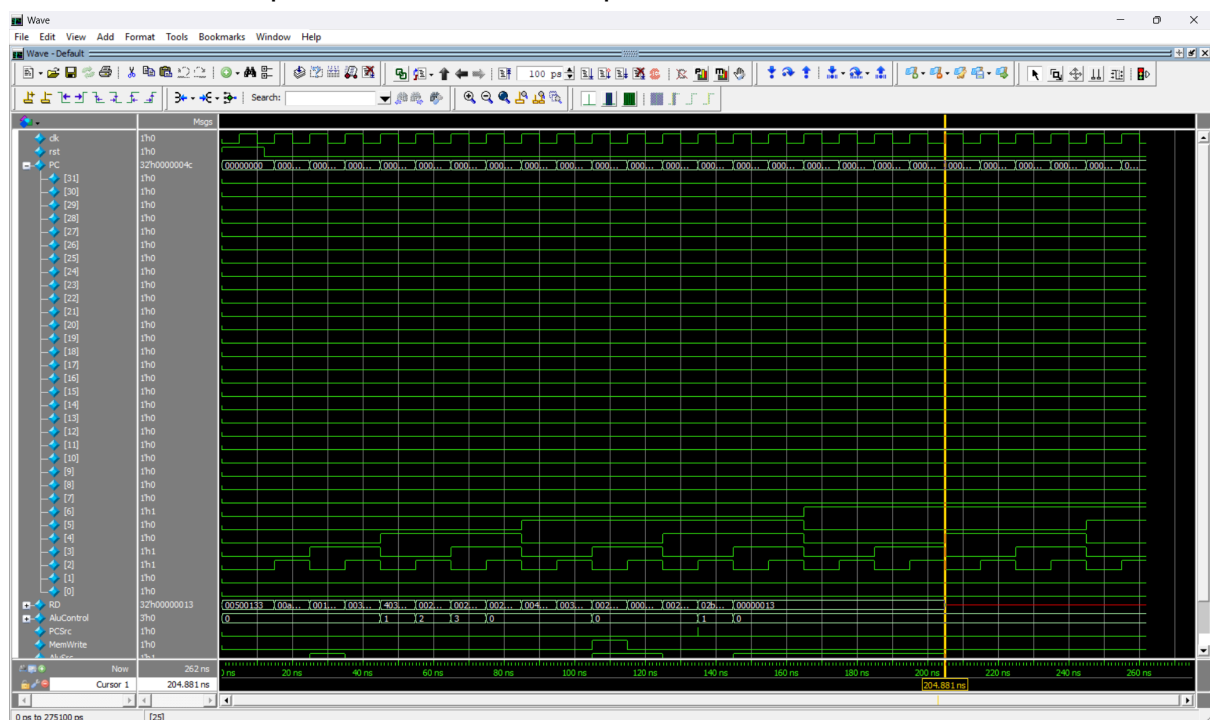
El programa de ejecución cargado dinámicamente valida la ruta de datos al implementar instrucciones aritméticas concurrentes que reutilizan registros consecutivos, forzando la actualización en flancos transicionales:

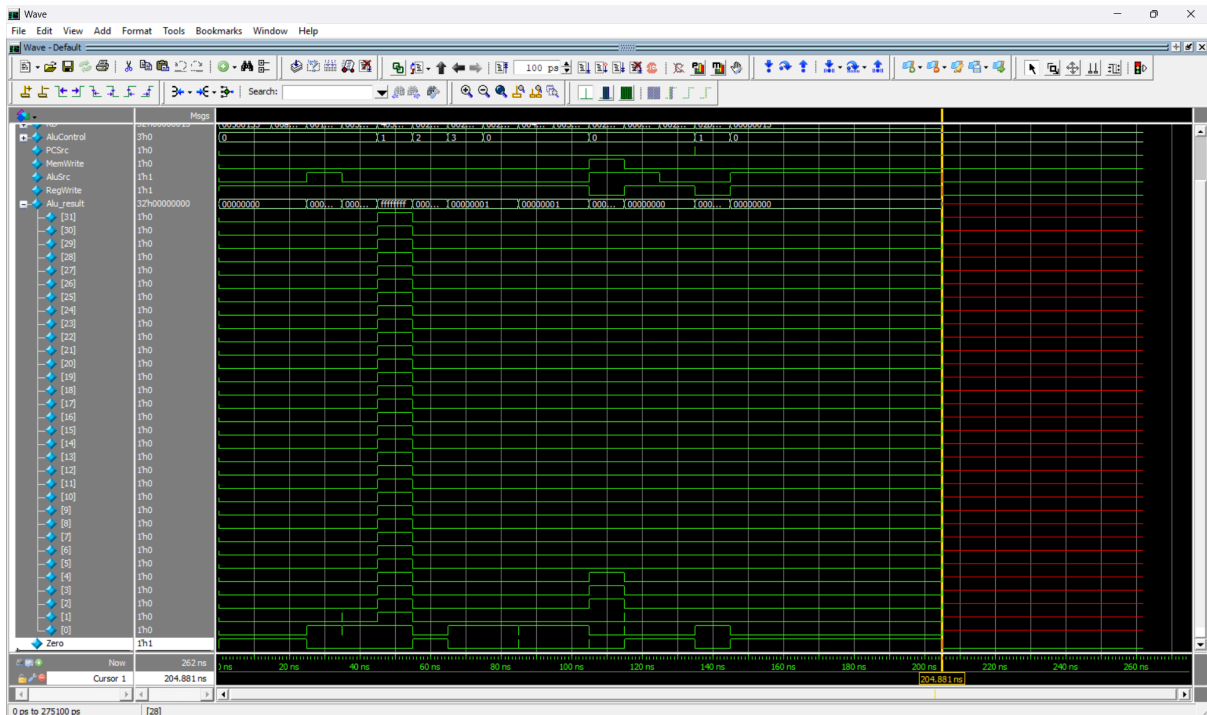
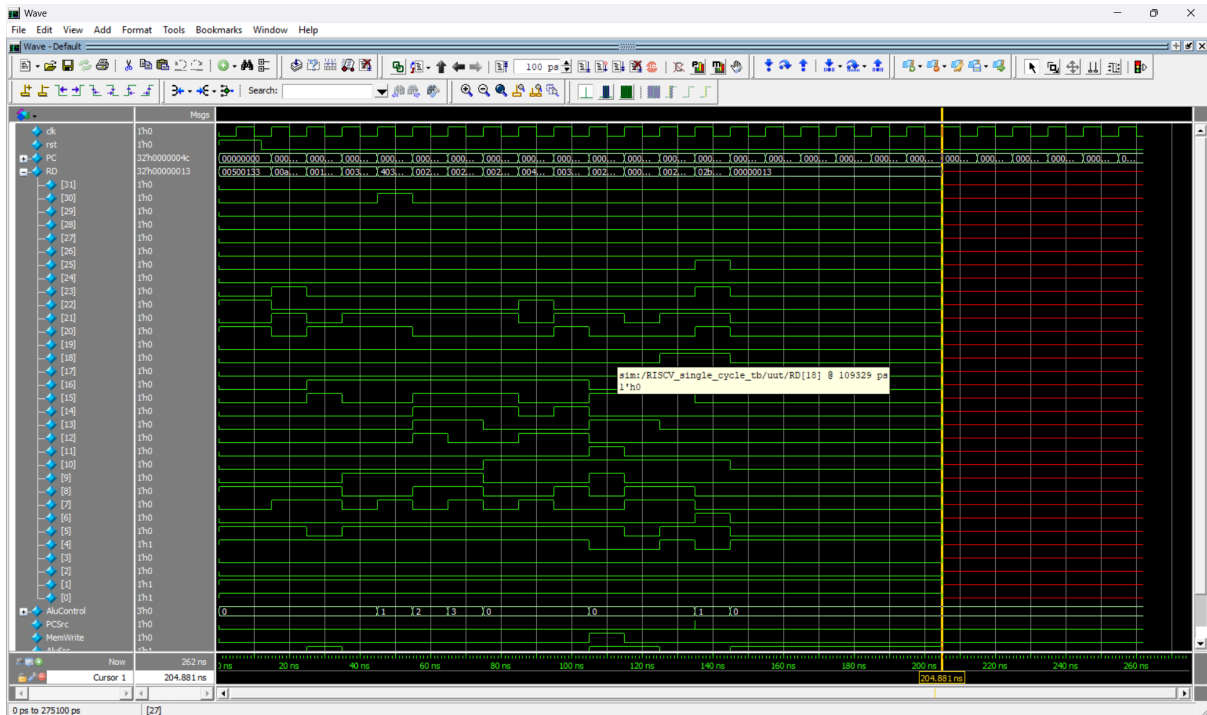
Dirección (PC)	Hexadecimal	Instrucción Ensamblador	Descripción y Efecto de Control Esperado
00000000	00500133	add x2, x0, x5	Suma $x0 + x5$ (15 inicial) y guarda en x2. RegWrite=1, AluSrc=0.
00000004	00a001b3	add x3, x0, x10	Suma $x0 + x10$ (25 inicial) y guarda en x3. RegWrite=1.
00000008	00118193	addi x3, x3, 1	Incrementa inmediato. $x3 = 25 + 1 = 26$.
0000000c	00310233	add x4, x2, x3	Reutilización inmediata de registro. Suma registros: $x4 = x2 + x3 = 15 + 26 = 41$ (0x29).

00000010	403102b3	sub x5, x2, x3	Resta registros: $x5 = 15 - 26 = -1$ (0xffffffff en compl. a 2).
00000014	0021f333	and x6, x3, x2	Operación lógica bitwise AND entre x3 y x2.
0000004c	00000013	addi x0, x0, 0 (nop)	Instrucción 20. Última operación válida del buffer de programa.

5. Evidencia de Simulación y Análisis de Resultados

El análisis del comportamiento obtenido a través del Test Bench e inspector de ondas de ModelSim (Questa) nos confirma la sincronía del diseño de ciclo único. A continuación se explican los resultados esperados:





En los resultados de la simulación observamos que el contador de programa (PC) avanza exactamente de 4 en 4 bytes (0x0, 0x4, 0x8...) con cada flanco de subida del reloj, que tiene un periodo de 10 ns. Esto demuestra que la sincronía del ciclo único funciona bien, ya que cada instrucción estabiliza el bus de la ALU y las señales de la Unidad de Control en un solo paso, sin retrasos ni errores de tiempo. En cuanto a la validación aritmética, el procesador hace la matemática exacta: por ejemplo, al

Llegar a la dirección $PC = 0x0000000c$, el resultado en la ALU es un **41** decimal ($0x29$ en hexadecimal), y al hacer la resta en $PC = 0x00000010$ ($\$15 - 26\$$), el sistema procesa correctamente el número negativo dando un **-1**, que en el visor de ondas aparece como $0xffffffff$ debido al formato de complemento a dos. Finalmente, lo que ocurre a los 210 ns es un comportamiento lógico y esperado: al intentar leer la instrucción 21 en la dirección $PC = 0x00000050$, la memoria ROM busca una línea que no existe en nuestro archivo programa.mem. Como el archivo solo tiene 20 líneas, la ROM entrega un valor desconocido (X) que, al no haber buffers que lo detengan, viaja de inmediato por todo el circuito combinacional pintando las ondas de color rojo. Esto no es una falla del procesador, sino una prueba clara de que el hardware reacciona y responde en tiempo real a lo que recibe de la memoria.

6. Conclusiones de Validación

El procesador cumple perfectamente con los tiempos de un diseño de ciclo único: la Unidad de Control traduce el código máquina en tiempo real y acomoda los caminos de datos mucho antes de que llegue el siguiente golpe de reloj. Las señales extrañas y las líneas rojas que se ven al final de la simulación no son un error de diseño en nuestro hardware, sino la consecuencia lógica de que se acabaron las instrucciones en el archivo de texto. Esto demuestra que el procesador responde exactamente como dicta la teoría clásica de arquitectura de computadoras.